Hedging against prediction error through inference delay in world-model planning

Runze Xia[1, *] and Richard Jiang[1, †]

[1] *School of Computing and Communications, Lancaster University, Lancaster, UK*

World-model planners take time to compute while a deployed system keeps moving. Predicting the state at which a plan takes effect is the standard remedy, but with learned world models it leaves most of the delay loss, because their errors through the delay are value-aligned. We propose residual-calibrated hedging, a training-free addition to sampling-based planners that scores each candidate over post-delay states formed from a world model's prediction and its own recorded errors. Scored in one batched call, four-anchor hedging incurred approximately 1–2% additional median latency in the tested configurations. In pre-registered tests on 24 recurrent, joint-embedding and TD-MPC2 world models, hedging over four states improved on predictive compensation at delays of four and eight decisions; sixteen states improved it only at eight. A three-model ensemble achieved higher returns, but its advantage was already present without delay. Residual hedging reduced the loss from prediction mismatch, including the ensemble's at the longer delay.

I. INTRODUCTION

Learned world models let an agent evaluate actions before taking them[1, 2]. In model predictive control[3, 4], a sampling-based planner proposes candidate action sequences, the model predicts their consequences, and the best sequence is executed, with planning repeated as observations arrive. The planner is typically model predictive path integral control (MPPI)[5, 6] or the cross-entropy method[7, 8], and this loop has been used with learned models from pixels[9–12]. Learned models also drive tree search[13] and policy learning in imagination[14, 15].

The models range widely in size, from compact recurrent state-space models[9] and joint-embedding predictors[16, 17] to video world models with hundreds of millions to billions of parameters[18–21]. Their planning calls take from milliseconds to minutes.

While a planner computes, a physical system keeps moving and executes actions that were committed earlier. Delayed decision processes have long been studied[22], and reinforcement learning has formalised the real-time setting[23–26] and benchmarked algorithms in it[27]. Control theory has a standard remedy: predict, with the model, the state at which the new control will be applied, and optimise from there[28–30]. The same remedy appears in delay-aware model-based reinforcement learning[31, 32], in model predictive control with learned models on robots[33], in tree search that simulates the committed steps[34] and in action-chunking policies[35, 36], as conditioning on the pending actions or on forecasts of the state at execution time[37–43]. We call it committed-prefix planning. With an exact model it loses nothing except replanning frequency[29].

Learned world models are not exact. Their multi-step predictions drift[44], and their training objective is not the planner's[45]. How much of the delay loss committed-prefix planning recovers with such models has not been measured. We measure it for three families of pixel world models on three control tasks, splitting the delay loss into a feedback-rate term and a prediction-mismatch term in the manner of networked predictive control[46]. Committed-prefix planning removes only about 30% of the mismatch. The remaining loss is not explained by the size of the prediction error: on the same states, a random error of the same size costs one-third to three-quarters as much as the model's actual error, so the actual errors fall along directions that matter for the plan's value. Corrections that assume small or low-dimensional errors do not help: neither a first-order sensitivity update[47, 48] nor bias removal.

We therefore propose residual-calibrated hedging (RCH), a training-free addition to any sampling-based planner that uses a learned world model. Before deployment, the model's own errors in predicting through committed actions are recorded on its training data. At each planning call, every candidate action sequence is scored by its average predicted cost over several post-delay states: the model's prediction, plus errors drawn from that record. A learned variant replaces the record with a small heteroscedastic error model conditioned on the state and the committed actions.

The idea of hedging over the execution-time state comes from robust and scenario-based predictive control[49, 50], with relatives in particle and tube formulations of sampling-based control[51, 52], chance-constrained MPPI under state-prediction uncertainty[53] and ensemble hedging in delay-aware reinforcement learning[31]. CLIS-MPPI[54] applied it to planning delay in MPPI: it optimises rollouts from sampled execution-time initial states and corrects the executed actions with rollout-based feedback, using a known robot model. RCH makes this kind of hedging work for learned world models, whose dominant uncertainty is model error rather than the length of the delay. The hedge distribution is calibrated from the model's own measured errors, not from an assumed delay distribution or an ensemble of probabilistic models[10, 31]. RCH therefore applies to any single pretrained world model, including determin-

* r.xia2@lancaster.ac.uk

† Contact author: r.jiang2@lancaster.ac.uk

istic latent models and video diffusion models. Compared directly with DATS-style hedging over three independently trained models, the ensemble achieved higher returns, but it did so with or without delay and lost as much to prediction mismatch as an unhedged single model; RCH reduced that loss, for a single model and, at the longer delay, for the ensemble. RCH also costs almost no latency: a single robot's planning call is limited by the sequential steps of the model rollout rather than by the number of candidate rows, so the hedge states can be scored in the same batched call.

Our contributions are threefold. The first is the method: residual-calibrated hedging, an algorithm and a deployment recipe for sampling-based planners with learned world models, with a learned heteroscedastic variant. The second shows why it is needed: with learned pixel world models, committed-prefix planning leaves about 70% of the mismatch loss, because the models' errors through the delay are value-aligned. The third is evidence from pre-registered tests on 24 world models in three families: four anchors improve on committed-prefix planning at delays of four and eight decisions, and sixteen anchors only at eight. A second study on held-out models and its ablations, where these gains are secondary comparisons, shows that the gain comes from hedging rather than from correcting the prediction or penalising uncertainty, and batched scoring keeps it at the latency of plain planning. A pre-registered comparison with ensemble hedging, with post-hoc analyses that separate model quality from delay compensation, shows that the two methods address different parts of the loss and combine at the longer delay.

II. RESULTS

A. Paused simulators hide the latency of world-model planners

Closed-loop evaluations of world-model planners almost always pause the simulator during planning[55–57]. Timing records from a configuration study of a diffusion world-model planner show what this can hide (Fig. 1). The planner is a MiniWorld-0.5B video model[18] fine-tuned on DeepMind Control Suite data and planned with MPPI[6]. Across six inference configurations, the amortised planning cost fell 3.41-fold, from 10.44 to 3.06 seconds per decision. The duration of one planning call fell only from 209 to 121 seconds, longer than the 10 to 25 seconds of simulated time in a whole episode. Planning less often halved the amortised cost and left the call duration unchanged. Throughput and latency are distinct measures of inference cost[58], and only latency determines how stale a plan is. Fewer sampler steps[59] shortened the call, but nowhere near a control period. Timed for a single environment on the same GPU, one call with two sampler steps took 28 s, about 1,400 cartpole decisions. Throughout this paper, latency means the duration of one planning call, and the delay d is the number of decisions that elapse during it.

B. Planning schedules and the decomposition of the delay loss

We compare four schedules (Fig. 2): the paused planner $\mathcal{P}_r$, where the simulator waits during planning and r actions are executed per plan; the naive planner $\mathcal{N}_d$, which executes a plan made for an earlier state from its first action; the time-aligned planner $\mathcal{A}_d$, which executes the same plan from position d; and committed-prefix planning $\mathcal{C}_d$. A run fixes the delay d, and the asynchronous planners $\mathcal{N}_d$, $\mathcal{A}_d$ and $\mathcal{C}_d$ make identical model calls (Methods).

With an exact deterministic model, $\mathcal{C}_d$ reproduces $\mathcal{P}_d$ under matched sampling noise. This is the nominal property of predictive delay compensation[29]; we verified it to 10^{-9} on an exact linear system. We therefore split the delay loss of an asynchronous planner $\mathcal{X}_d$, with R the normalised return (Methods), into a feedback-rate loss and a mismatch loss,

$$\begin{aligned} F(d) &= R(\mathcal{P}_1) - R(\mathcal{P}_d), \\ M(d) &= R(\mathcal{P}_d) - R(\mathcal{X}_d). \end{aligned} \tag{1}$$

Because $\mathcal{C}_d$ reproduces $\mathcal{P}_d$ with an exact model, $M(d) = R(\mathcal{P}_d) - R(\mathcal{C}_d)$ measures prediction mismatch alone.

C. Committed-prefix planning leaves a residual mismatch

We ran recurrent state-space (RSSM)[9], joint-embedding predictive (JEPA)[16] and TD-MPC2[12] world models on cartpole swing-up, walker walk and cup catch[55], with three independently trained models per family and task. Each model was evaluated under delays of $d \in \{1, 2, 4, 8, 16\}$ decisions, with 50 episodes per condition starting from states shared across conditions, and planned with MPPI (population 512, 6 iterations, horizon 32 decisions; Methods).

Committed-prefix planning beat time-aligned planning at every delay, and the margin grew with the delay. Pooled over the eight family–task cells above the prespecified return floor, the margin rose from +0.010 normalised return at $d = 1$ to +0.048 [+0.028, +0.068] at $d = 16$ (Fig. 3a). The pre-registered test that the margin is positive at each of $d = 2, 4, 8$ just failed, with $p = 0.050$. The naive planner was far worse than either.

Splitting the delay loss shows where the return goes (Fig. 3b). Up to $d = 8$, most of it is mismatch rather than reduced feedback. At $d = 8$, the time-aligned planner loses 0.133 [0.104, 0.163] to mismatch and 0.080 [0.047, 0.115] to replanning less often. Committed-prefix planning reduces the mismatch to 0.092 [0.072, 0.114],

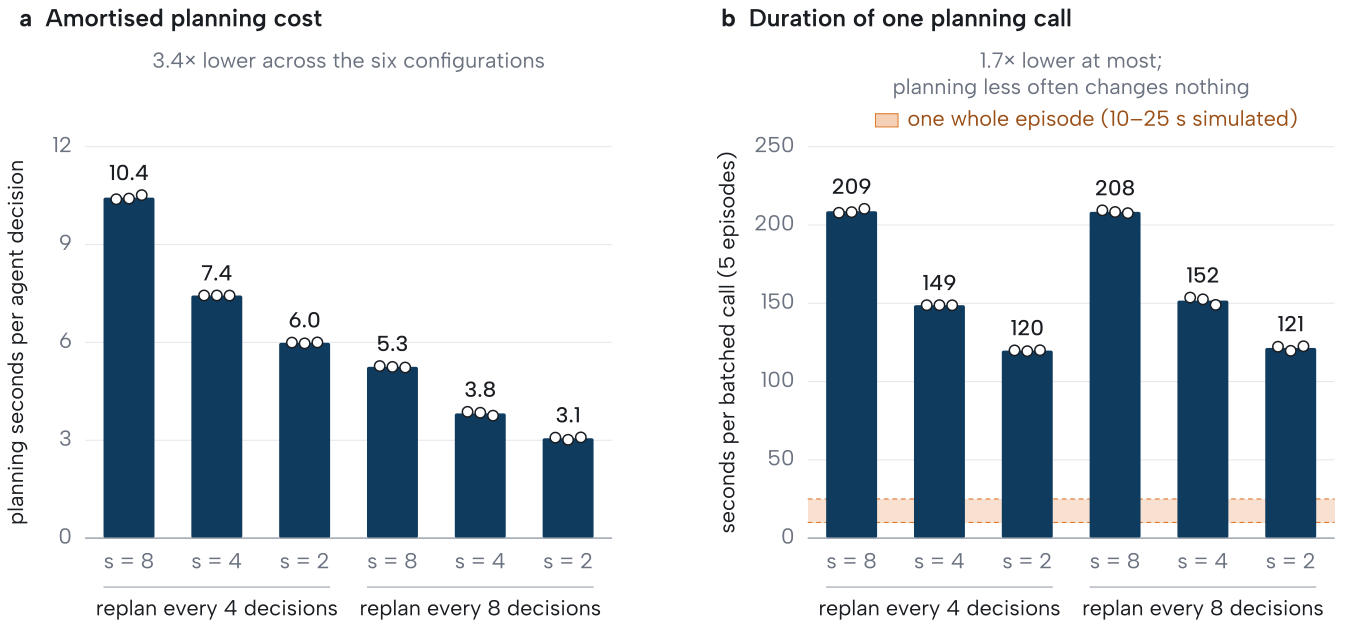


FIG. 1. **Amortised planning cost and the duration of a planning call measure different things.** Reanalysis of an 18-job configuration study of a diffusion world-model planner on one V100 GPU (three DeepMind Control tasks, five episodes per task and configuration, simulator paused while planning). Bars are grouped by the number of decisions r served by one planning call, and s is the number of sampler steps. Bars show the mean over the three tasks and white dots the individual tasks (left to right: cartpole swing-up, walker walk, cup catch). **a**, Planning seconds per agent decision. **b**, Mean duration of one planning call serving five batched episodes, reconstructed from accumulated timers and the number of calls (Methods). The shaded band marks the simulated duration of a whole episode (10–25 s). No timing intervals are shown because independent timing repetitions were not recorded.

which removes 31% of it. Across delays it removes 27–36%, against the pre-registered expectation of at least half.

D. The residual comes from value-aligned prediction errors

To find what limits committed-prefix planning, we replayed held-out data windows with each model, used the recorded actions as the committed prefix, and planned once from the predicted post-delay latent and once from the true filtered latent, with identical sampling noise (Methods). Regret is the model-predicted cost of the plan from the predicted latent, evaluated from the true latent, minus that of the plan made from the true latent. Replanning from the true latent with different noise gives regret near zero, so the regret reflects the anchor rather than sampling variance.

The predicted latent was close to the truth: its error was under 20% of the stale error for the RSSM and JEPA models, and 7–140% for TD-MPC2. Regret was nonetheless large wherever committed-prefix planning had failed in closed loop (Table I). Replacing the model's error by a random error of the same size reduced regret by a factor of 1.3 to 3, so the actual errors fall along value-relevant directions. Halving the error roughly halved the regret. Two corrections failed: a finite-difference sensitivity correction along the eight leading directions of calibration residuals, which held only 10–50% of the error, and removing the mean residual. Selecting, on arrival, the plan of whichever of 16 candidate anchors lay nearest the observed state also barely helped. Scoring each candidate by its average cost over anchors drawn from calibration residuals reduced regret by roughly a quarter to a half wherever there was regret to remove (more in one noisy case).

E. Residual-calibrated hedging

Residual-calibrated hedging keeps the committed-prefix schedule and changes only how a candidate is scored (Algorithm 1 in Methods).

Calibration, once per model and delay. The model's own d-step prediction residuals $r = z_{t+d} - \hat{z}_{t+d}$ are recorded on training windows, giving a bank of 512 residuals. The learned variant instead fits a small network to the same residuals, conditioned on $\hat{z}_{t+d}$, z_t and the committed actions; it predicts the residual's mean and per-dimension variance (Methods).

At each call. The committed actions are rolled through the model once, giving the predicted latent $\hat{z}$.

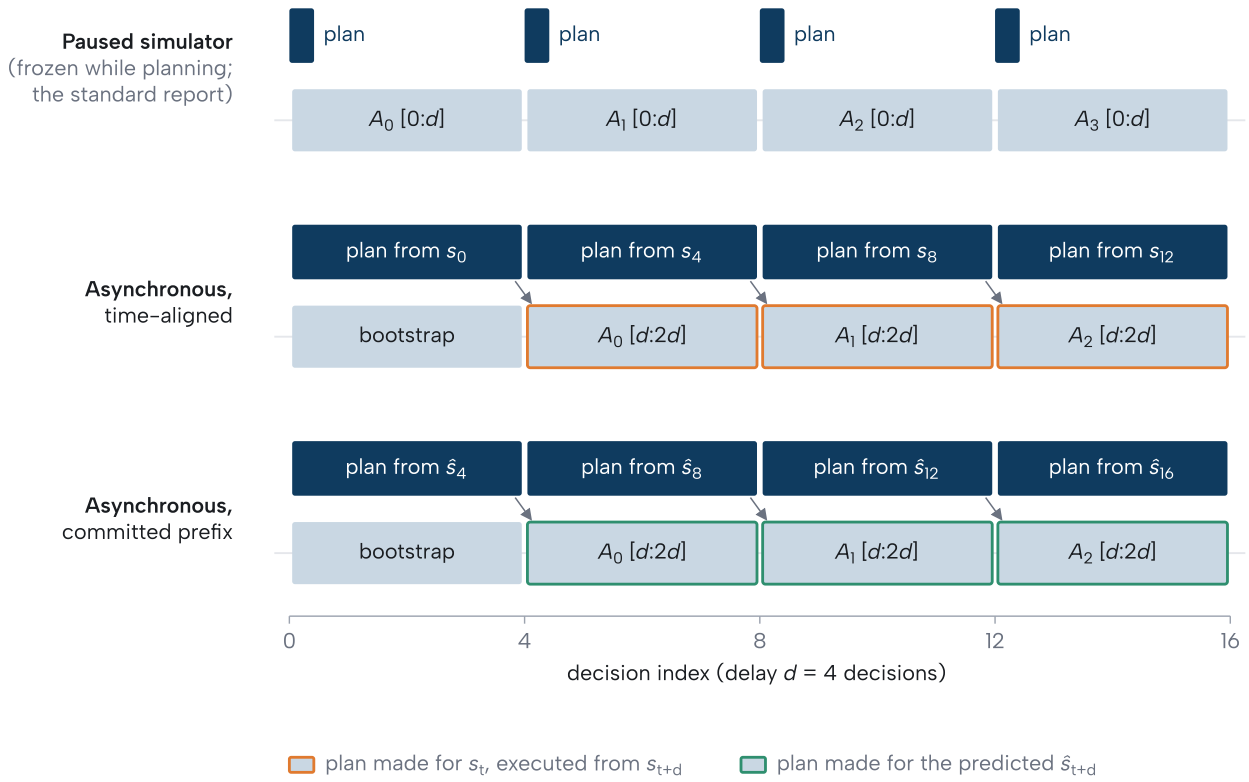


FIG. 2. **Planning schedules under a delay of $d = 4$ decisions.** Top, the paused reference, in which the simulator is frozen during each plan. Middle, time-aligned asynchronous planning: each call plans from the observation at its start and its actions from position d onward are executed after the call returns, from a state the plan did not assume (orange outline). Bottom, committed-prefix planning: each call rolls the world model through the actions that will execute during the call and plans from the predicted state $\hat{s}_{t+d}$ (green outline). Residual-calibrated hedging keeps this schedule and changes how candidates are scored (Algorithm 1 in Methods).

TABLE I. Anchor regret on held-out windows at $d = 2$

Model	Stale latent	Predicted latent	Random error, same size	Half the error	Hedged, $K = 16$
RSSM walker	16.28	7.84	3.90	3.94	3.59
JEPA cartpole	0.58	0.45	0.14	0.18	0.32
TD-MPC2 walker	6.08	3.65	2.72	2.94	2.77
TD-MPC2 cup	2.68	1.65	0.82	0.42	−1.55
RSSM cup	2.95	−0.02	0.00	−0.03	−0.03

Model-predicted cost from the true post-delay latent of the plan made from each anchor, minus that of the plan made from the true latent (model cost units, 64 windows). Hedged: one plan scored by the mean cost over 16 anchors, the predicted latent plus 15 calibration residuals. RSSM cup is a control where committed-prefix planning already works.

K anchors are formed: $\hat{z}$ itself, and $\hat{z}$ plus $K - 1$ residuals drawn from the bank; in the learned variant, the anchors are the corrected mean and $K - 1$ draws from the predicted distribution. Each candidate's free actions are scored by the mean predicted cost over the anchors, using the same samples for every anchor, in one batched call.

With $K = 1$ the method is committed-prefix planning. The hedge targets the failure diagnosed above: a plan tuned to one predicted state is fragile when the error falls in value-relevant directions, and averaging over plausible post-delay states favours plans that are good across them.

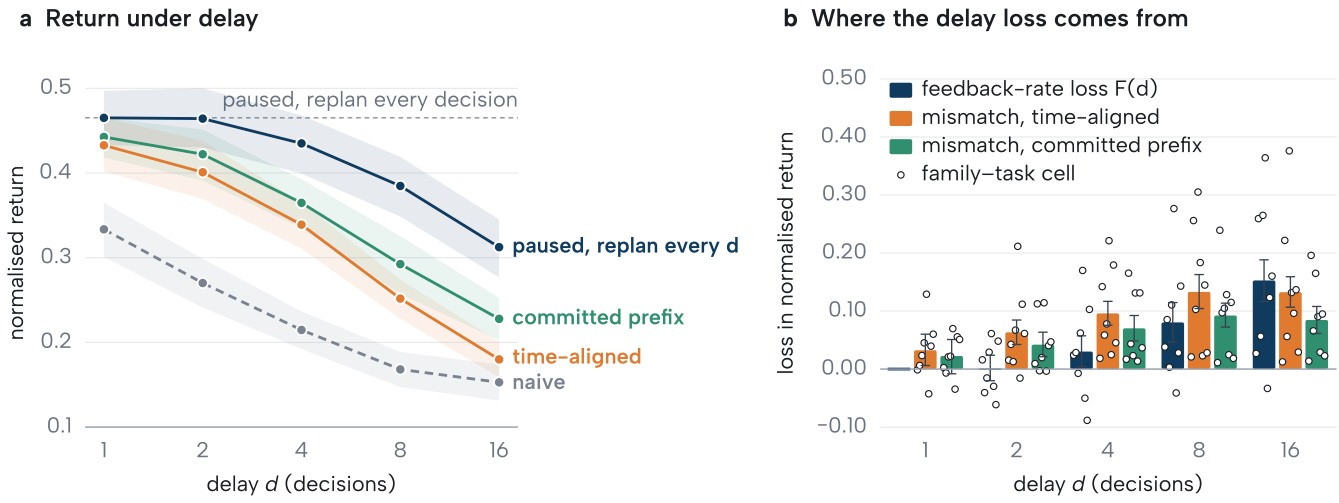


FIG. 3. **Committed-prefix planning recovers a minority of the delay loss with learned world models.** RSSM, JEPA and TD-MPC2 world models on three DeepMind Control tasks, eight family–task cells above the return floor, three models per cell, 50 episodes per model and condition. **a**, Normalised return against delay for each schedule; the dashed line marks the paused planner that replans every decision. Bands are 95% hierarchical bootstrap intervals of each schedule's pooled mean; paired contrasts between schedules are tighter (main text). **b**, Delay loss of the asynchronous planners split into feedback-rate loss $F(d) = R(\mathcal{P}_1) - R(\mathcal{P}_d)$ and mismatch loss $M(d) = R(\mathcal{P}_d) - R(\mathcal{X}_d)$. Bars are equal-weight means over the eight cells, white dots show the individual cells (three models and 50 episodes per model each), and error bars are 95% hierarchical bootstrap intervals of the pooled mean. $F(1)$ is zero by definition.

F. Four-anchor hedging recovers return at delays of four decisions or more

Over all 24 models, the pre-registered test of 16 anchors against committed-prefix planning confirmed a gain at $d = 8$ but not at $d = 4$ (Table II). At $d = 8$ the gain was $+0.036$ $[+0.014, +0.057]$ normalised return (Holm-adjusted one-sided $p = 0.0014$); at $d = 4$ it was $+0.008$ $[-0.010, +0.027]$ ($p = 0.19$), so the pilot's marginal gain did not replicate. The pre-registered equal-latency test of four anchors, which add no latency for a single environment (see 'Hedging costs almost no latency for a single robot'), succeeded at both delays: $+0.022$ $[+0.003, +0.040]$ at $d = 4$ (Holm-adjusted one-sided $p = 0.035$) and $+0.037$ $[+0.016, +0.058]$ at $d = 8$ ($p = 0.0004$). Neither number of anchors helped at $d = 2$, where little mismatch remains.

Against time-aligned planning, the common asynchronous baseline, four-anchor hedging doubled the gain of committed-prefix planning (secondary contrasts, Table III).

In the single-model pilot, gains were largest where diagnosis had found value-aligned errors, for example JEPA cup catch at $d = 8$ (0.43 to 0.68 with 16 anchors) and RSSM walker (0.13 to 0.17). Hedging hurt slightly on JEPA cartpole and on TD-MPC2 cartpole and walker.

G. The gain holds on held-out models, and it comes from hedging

A second pre-registered study tested the learned variant and two ways of steering the planner with the error model. Penalty strengths were tuned on the seed-0 models, using episodes from a separate seed range. All contrasts were then estimated on the 16 seed-1 and seed-2 models, which were never used to design or tune anything, with $K = 4$, population 512, $d \in \{4, 8\}$ and 50 episodes per model (Table IV). Four ablations locate the gain (Methods): mean correction only, with one anchor at the corrected mean; a fixed spread, with draws using the global residual standard deviation instead of the predicted one; a hand-off penalty on the predicted uncertainty of the next call's anchor; and a risk penalty on the standard deviation of the cost over anchors.

Both forms of hedging again improved on committed-prefix planning. The residual bank gained $+0.025$ at $d = 4$ and $+0.037$ at $d = 8$; the learned error model gained $+0.025$ and $+0.047$. Against time-aligned execution, the learned variant gained $+0.044$ and $+0.090$. The learned model did not beat the residual bank, but at $d = 8$ its state-dependent spread beat a fixed spread of the same average size. Correcting the prediction by the learned mean alone gave smaller gains whose intervals include zero.

Two ways of steering the planner failed. A penalty on the predicted uncertainty of the next hand-off lowered return at both delays, at the tuned strength: it

TABLE II. Closed-loop contrasts over 24 models

Family	Contrast	d	Difference [95% CI]	p (Holm)
Replication	$K = 16$ vs $K = 1$, $N = 512$	4	$+0.008$ $[-0.010, +0.027]$	0.19
		8	$+0.036$ $[+0.014, +0.057]$	0.0014
Equal latency	$K = 4$, $N = 512$ vs $K = 1$, $N = 512$	4	$+0.022$ $[+0.003, +0.040]$	0.035
		8	$+0.037$ $[+0.016, +0.058]$	0.0004
	$K = 16$, $N = 128$ vs $K = 1$, $N = 512$	4	-0.034 $[-0.053, -0.015]$	1.0
		8	-0.038 $[-0.061, -0.015]$	1.0
Secondary	$K = 16$ vs $K = 1$, $N = 512$	2	-0.006 $[-0.026, +0.013]$	–
	$K = 4$ vs $K = 1$, $N = 512$	2	-0.001 $[-0.019, +0.017]$	–

Difference in normalised return, pooled with equal weight over eight family–task cells (three models each, 50 episodes per model and arm, shared initial states). 95% hierarchical bootstrap intervals; one-sided p, Holm-adjusted within each pre-specified family (Methods). K: anchors; N: population. All arms use 6 MPPI iterations and horizon 32.

TABLE III. Gains over time-aligned planning (secondary contrasts)

	$d = 4$	$d = 8$
Committed prefix vs time-aligned	$+0.020$	$+0.035$
Hedged ($K = 4$) vs time-aligned	$+0.042$ $[+0.024, +0.060]$	$+0.072$ $[+0.047, +0.098]$

Difference in normalised return over the same 24 models as Table II, with 95% hierarchical bootstrap intervals for the hedged planner.

drives the planner away from fast motions that are hard to predict, such as catching. A generic mean–standard-deviation risk penalty did not add to hedging.

The four primary tests of this study concerned the learned variant and the hand-off penalty, and none succeeded (Methods). The gains over committed-prefix planning are secondary contrasts here, and they agree with the pre-registered four-anchor result of Table II. The gain therefore comes from hedging itself. The residual bank is the simplest form to adopt; the learned model is an option when a calibrated, state-dependent spread is wanted.

H. Hedging costs almost no latency for a single robot

The delays studied here are the ones these planners incur. We timed the default planner (population 512, 6 iterations, horizon 32) for one environment on a node held exclusively, using the 95th percentile of 200 calls (Table V; protocol in Methods). The H200 is slower than the L40S here because a single environment's call is a long sequence of small kernels rather than a large computation.

A single-environment planning call is limited by the sequential steps of the model rollout, not by the number of candidate rows. Scoring all anchors in one batched call therefore uses GPU width that is otherwise idle (Table VI). For all three families on an L40S GPU, hedging over four anchors at population 512, or over 16 anchors at population 128, kept the call within 3% of plain committed-prefix planning at population 512. Sixteen anchors at population 512 cost 32–55% more.

Holding latency fixed shows which form to use (Table II). Four anchors at population 512, within 2% of the plain latency, beat committed-prefix planning at $d = 4$ and $d = 8$ (Holm-adjusted $p = 0.035$ and 0.0004). Sixteen anchors at population 128, equally fast, did worse (-0.034 and -0.038): shrinking the population to make room for anchors costs more than the hedge returns.

I. An ensemble improves the model, and residual hedging the handling of delay

DATS-style ensemble hedging[31] is the closest learned-model alternative to residual-calibrated hedging. We therefore compared the two directly, on the same eight cells, delays and initial states (Methods). The ensemble consisted of the three independently trained models of each cell. Each member filtered its own latent, predicted through the committed actions with its own dynamics and scored the shared candidates, and candidate costs were averaged over members. We ran it at the population and iterations of RCH, and at two iterations, which matches the latency of single-model planning when the members are evaluated in turn (Table VIII).

In the pre-registered comparison, RCH did not match the ensemble (Table VII). RCH was below the latency-matched ensemble at $d = 4$: -0.042 $[-0.070, -0.016]$, Holm-adjusted two-sided $p = 0.0008$. The other three differences also favoured the ensemble, but their intervals included zero. Against single-model committed-prefix planning, the ensemble gained $+0.046$ and $+0.045$ at $d = 4$ and $d = 8$ at matched population, and $+0.064$ and $+0.063$ at matched latency; RCH gained $+0.022$ and $+0.037$. The fewer iterations do not explain the latency-matched result: in a post-hoc check on all 24 models, RCH scored the same at two iterations as at six (-0.003

TABLE IV. Held-out comparisons and ablations

Contrast	$d = 4$	$d = 8$
Residual bank vs committed prefix	$+0.025$ $[+0.003, +0.047]$	$+0.037$ $[+0.011, +0.064]$
Learned error model vs committed prefix	$+0.025$ $[+0.006, +0.046]$	$+0.047$ $[+0.013, +0.082]$
Learned error model vs time-aligned	$+0.044$ $[+0.025, +0.063]$	$+0.090$ $[+0.063, +0.116]$
Learned error model vs residual bank	$+0.001$ $[-0.018, +0.019]$	$+0.011$ $[-0.008, +0.030]$
Mean correction only vs committed prefix	$+0.015$ $[-0.006, +0.038]$	$+0.020$ $[-0.008, +0.049]$
Fixed spread vs learned spread	$+0.008$ $[-0.010, +0.026]$	-0.020 $[-0.039, -0.002]$
Hand-off penalty vs none	-0.026 $[-0.045, -0.007]$	-0.021 $[-0.040, -0.003]$
Risk penalty vs none	-0.016 $[-0.039, +0.006]$	-0.027 $[-0.057, +0.001]$

Sixteen seed-1 and seed-2 models in eight cells, $K = 4$ anchors, population 512, 50 episodes per model and arm; difference in normalised return with 95% hierarchical bootstrap intervals. Penalty strengths were tuned on the seed-0 models with separate episodes ($\lambda = 0.25$ for the hand-off penalty, 0.5 for the risk penalty).

TABLE V. Single-environment latency of the default planner and the delay it implies

GPU	Call time	Delay on cartpole	Delay on walker and cup catch
L40S	77–98 ms	4–5 decisions	2–3 decisions
H200	103–137 ms	6–7 decisions	3–4 decisions
V100	126–161 ms	7–9 decisions	3–4 decisions

Population 512, 6 iterations, horizon 32; 95th percentile of 200 calls on a node held exclusively.

and -0.005).

Two post-hoc analyses show where the ensemble's advantage comes from. First, it does not depend on delay. With the simulator paused and a new plan at every decision, the ensemble scored $+0.066$ $[+0.033, +0.099]$ above the single models, more than its gain under delay. Second, it does not reduce the mismatch loss. We measured each planner's mismatch loss against the same planner with the simulator paused and the same replanning interval. On this measure, the ensemble lost as much to prediction mismatch as single-model committed-prefix planning (Table VII, bottom). Its mismatch loss minus that of committed-prefix planning was 0.000 $[-0.023, +0.023]$ at $d = 4$ and $+0.012$ $[-0.011, +0.036]$ at $d = 8$. RCH lost less to mismatch than the ensemble, by 0.022 $[-0.004, +0.049]$ at $d = 4$ and 0.049 $[+0.024, +0.075]$ at $d = 8$. Adding RCH to the ensemble cut the ensemble's mismatch loss at $d = 8$ from 0.111 to 0.078, a secondary contrast of $+0.033$ $[+0.008, +0.058]$ in return, but not at $d = 4$. Averaging independently trained models gives a better model, hedging over the model's recorded errors gives better handling of delay, and at the longer delay the two combine.

On the RSSM, the one family with a stochastic transition, hedge states sampled from the model's own transition matched RCH at $d = 8$. At $d = 4$ they exceeded it: $+0.020$ $[+0.002, +0.036]$, three cells. A model with a calibrated stochastic transition can therefore supply its own hedge states; RCH supplies them for deterministic models.

The ensemble's gains need three trained models (Table VIII). That triples the training, the weights and the per-decision filtering. At matched population the call takes 2.9 times as long; at matched latency the ensemble runs a third of the iterations. RCH needs one model and a residual bank recorded in minutes. For a pretrained video world model such as MiniWorld-0.5B, each further member would be another half-billion-parameter model to train and serve.

J. Fixed-delay evaluation matches real-time control when the delay covers the whole loop

All results above use a fixed delay per run. To check this protocol, we ran the RSSM on cartpole in real time on a V100 GPU (Methods). A simulator process advanced one decision every 20 ms of wall time; the planner, in a separate process, filtered each new frame and planned; and the committed prefix was set from a latency measurement taken at start-up.

With a cheaper planner (population 128, 2 iterations), calls took a median of 4 decisions against an estimate of 5. Real-time returns matched the fixed-delay simulation: committed-prefix planning scored 0.378 ± 0.014 in real time, against 0.350 ± 0.019 and 0.337 ± 0.011 simulated at $d = 4$ and $d = 8$ (mean $\pm$ s.e.m., 30 episodes each).

With the default planner, the start-up estimate of 11 decisions left out the time spent filtering frames inside the loop. Calls took 14 decisions, so the committed prefix ended three decisions early, and committed-prefix planning fell below time-aligned execution (0.227 ± 0.012 against 0.287 ± 0.008, mean $\pm$ s.e.m., 30 episodes each). Compensation by prediction, hedged or not, needs the delay of the whole control loop, with margin.

TABLE VI. Single-environment latency of hedged planning

Configuration	Rows per call	RSSM walker	JEPA cup	TD-MPC2 cup
$K = 1$, population 512 (committed prefix)	512	102 ms	95 ms	81 ms
$K = 4$, population 512	2,048	103 ms	96 ms	83 ms
$K = 16$, population 128	2,048	105 ms	98 ms	84 ms
$K = 16$, population 512	8,192	158 ms	125 ms	112 ms

Median of 50 calls with CUDA synchronisation, L40S GPU, 6 MPPI iterations, horizon 32, $d = 4$, anchors scored in one batched call, timed on a shared node. Plain-planner latencies on exclusive nodes are given in Table V.

TABLE VII. Residual-calibrated hedging and ensemble hedging

	$d = 4$	$d = 8$
Normalised return		
Committed prefix, one model	0.359	0.286
RCH ($K = 4$), one model	0.381	0.323
Ensemble of three, matched population	0.405	0.332
Ensemble of three, matched latency	0.423	0.349
Ensemble with RCH	0.402	0.365
RCH minus ensemble (pre-registered)		
Matched population	-0.023 $[-0.051, +0.002]$	-0.008 $[-0.038, +0.021]$
Matched latency	-0.042 $[-0.070, -0.016]$	-0.026 $[-0.057, +0.004]$
Mismatch loss, $R(\mathcal{P}_d) - R(\text{planner})$ (post hoc)		
Committed prefix, one model	0.076	0.098
RCH ($K = 4$), one model	0.054	0.061
Ensemble of three, matched population	0.076	0.111
Ensemble of three, matched latency	0.068	0.103
Ensemble with RCH	0.079	0.078

Eight family–task cells, 50 episodes per arm on shared initial states. Single-model rows average the three models of each cell; ensemble rows use the three together. Population 512 and 6 iterations, except the matched-latency ensemble (2 iterations). Contrasts: 95% hierarchical bootstrap intervals; Holm-adjusted two-sided $p = 0.21$ and 0.60 (matched population) and 0.0008 and 0.21 (matched latency) at $d = 4$ and 8. Mismatch loss is measured against the same planner, one model or the ensemble, with the simulator paused and a new plan every d decisions, $\mathcal{P}_d$.

K. A video diffusion world model

For one environment on a V100 GPU, a planning call of the diffusion world model takes 28–44 s without a prefix (2 to 8 sampler steps), and 42–58 s with a 16-decision prefix. Its real delay therefore exceeds an episode, and the delays studied here correspond to hardware roughly two orders of magnitude faster. In closed loop, with 10 episodes per arm, the diffusion results do not separate the planners (Table IX). Committed-prefix planning beat time-aligned planning on cup catch at $d = 8$ (0.19 against -0.01), but not on cartpole (0.27 against 0.30). With episodes paired, four anchors against one gave (mean paired difference $\pm$ s.e.m.) $+0.18 \pm 0.09$ on cup catch at $d = 8$ (0.30 against 0.12), $+0.01 \pm 0.03$ on cartpole at $d = 8$, $+0.08 \pm 0.08$ on cup catch at $d = 16$ and -0.05 ± 0.02 on cartpole at $d = 16$.

Cup catch is close to all-or-nothing for this model, so its standard errors are large. At $d = 16$ the time-aligned planner already loses little to mismatch on cartpole (0.23 against 0.26 for the paused planner), which leaves little for hedging to recover. We therefore treat the diffusion results as a feasibility demonstration, not as evidence for or against hedging.

III. DISCUSSION

Predicting the state at which a plan takes effect is the standard response to inference delay[29, 33, 34]. With learned pixel world models it recovers only part of the delay loss. The remainder is not a matter of error size alone: the models' errors through the committed actions point in value-relevant directions, so plans tuned to a single predicted state are fragile. Residual-calibrated hedging addresses this without retraining, and batched scoring makes it nearly free at deployment batch size. In pre-registered tests across 24 models, hedging over four anchors improved on committed-prefix planning at delays of four and eight decisions at no latency cost, and in secondary comparisons the gain held on held-out models. It did not help at two decisions, where little mismatch remains. Sixteen anchors bought with a smaller population did worse. Correcting the prediction, or penalising predicted uncertainty, did not help: what matters is scor-

TABLE VIII. Cost of ensemble hedging

	RSSM walker	JEPA cup	TD-MPC2 cup
Call latency, one environment			
Committed prefix, one model	103 ms	94 ms	83 ms
RCH ($K = 4$), one model	103 ms	95 ms	83 ms
Ensemble of three, 6 iterations	299 ms	272 ms	239 ms
Ensemble of three, 2 iterations	100 ms	91 ms	80 ms
Ensemble with RCH, 6 iterations	303 ms	277 ms	242 ms
Particle anchors ($K = 4$), one model	107 ms	–	–
Per model			
Parameters	9.3 M	3.3 M	7.8 M
Weights in GPU memory	35 MiB	13 MiB	30 MiB
Latent filtering per decision	0.77 ms	0.49 ms	0.46 ms
Training (V100 GPU-hours)	2.2	1.4	0.8

Latency: median of 50 calls with CUDA synchronisation, L40S GPU on a shared node, $d = 4$, population 512, horizon 32, with the configurations timed in alternation; the ensemble evaluates its members in turn. Peak planning memory above the weights was 10–18 MiB for one model or the ensemble, and 28–46 MiB with four anchors. Filtering three latents took 1.33–2.23 ms per decision. Training is the wall time of the original training jobs. Each ensemble member is a separately trained model, so the ensemble triples parameters, weight memory, filtering and training.

TABLE IX. Diffusion world model under delay

Task	d	Paused	Time-aligned	Committed prefix	Hedged, $K = 1$	Hedged, $K = 4$
Cartpole	8	0.34 ± 0.02	0.30 ± 0.02	0.27 ± 0.02	0.28 ± 0.02	0.29 ± 0.02
Cup catch	8	0.16 ± 0.10	-0.01 ± 0.04	0.19 ± 0.11	0.12 ± 0.12	0.30 ± 0.14
Cartpole	16	0.26 ± 0.01	0.23 ± 0.01	0.25 ± 0.01	0.30 ± 0.02	0.25 ± 0.01
Cup catch	16	0.02 ± 0.08	-0.06 ± 0.00	0.04 ± 0.10	-0.06 ± 0.00	0.02 ± 0.08

MiniWorld-0.5B, 2 sampler steps, population 64, 2 iterations; normalised return, mean ± s.e.m. over 10 episodes with shared initial states. One anchor is committed-prefix planning with the prefix sampled once per episode; the committed-prefix column samples it per candidate.

ing plans against the plausible states, not choosing one of them or avoiding them. A three-model ensemble achieved higher returns than RCH, significantly so at matched latency at $d = 4$. Its advantage was present without delay, though, and it did not reduce the mismatch loss; RCH did, and it added to the ensemble at $d = 8$. With the video diffusion model, ten episodes per arm could not separate the planners.

The closest method is CLIS-MPPI[54]. It samples execution-time initial states from a distribution induced by the planning delay, optimises rollouts from them, and corrects the executed actions with rollout-based feedback; it was demonstrated in five real runs on a manipulator with a known model, where it avoided collisions that standard MPPI incurred. Residual-calibrated hedging differs in three ways. Its hedge states come from a learned world model's own prediction errors, recorded once or modelled by a small post-hoc network. It targets planners whose model, not the delay length, is the main source of uncertainty, and it needs no model of the robot. And it is evaluated across three model families with pre-registered tests and a study on held-out models. Three other lines of work are complementary: sensitivity-based updates[47, 48], learned corrections of queued actions[60], and re-anchoring on newer observations between iterations, as in the real-time iteration scheme[61, 62]. A sensitivity update did not help in our diagnosis, because the errors are not low-dimensional; we did not test the other two. Post-hoc correction of the post-delay state[42], the closest learned alternative, corresponds to our mean-correction ablation, which gave smaller gains.

DATS[31] hedges with an ensemble instead. In our comparison, ensemble hedging behaved as model averaging: it raised return by a similar amount with and without delay, and its committed-prefix planner lost as much to mismatch as a single model's. One possible explanation, which we did not test, is that members trained on the same data share much of their error through the delay, so their disagreement understates the error of any one of them. RCH draws instead on each model's measured errors. The ensemble also needs three trained models, three times the per-decision filtering and, at matched population, 2.9 times the call latency. Residual-calibrated hedging is therefore the option for a single pretrained model, and a complement to an ensemble where one is available.

Several limitations apply. A conditional error model

did not hedge better than the unconditional bank overall; its advantage appeared only against a fixed spread at the longer delay. Richer error models, or conformal calibration of their spread, may do better. The gains are confined to longer delays and differ by model and task; at $d = 2$, hedging did not help. The environments are simulated and deterministic, and apart from one real-time check the delay is fixed per run. The anchor count K was chosen from two values, and the population was not retuned for hedging. The ensemble comparison used one ensemble of three members per cell, so its intervals omit variation between ensembles. The analyses that separate model quality from delay compensation were post hoc. Larger ensembles, or members trained on different data, may hedge better. Predictability penalties hurt here, but we tried one form and one tuned strength. The diffusion model's real latency exceeds an episode on current hardware, so its results concern moderate delays that faster hardware would bring, and its closed-loop evaluation is too small (10 episodes per arm, one model) to test hedging.

Adding CLIS-MPPI's feedback correction[54] to residual anchors is a natural next step. Adaptive replanning[63] and faster diffusion samplers[59, 64] would shorten the delay itself, and hedging can be combined with either.

IV. METHODS

A. Environments and tasks

We use DeepMind Control Suite tasks[55], simulated in MuJoCo[65]: cartpole swing-up, walker walk and cup catch are run with action repeat 2 and 500 decisions per episode, which gives decision periods of 0.02, 0.05 and 0.04 s. Returns are normalised as $(R - R_{\text{rand}})/(R_{\text{exp}} - R_{\text{rand}})$ with reference pairs (880, 49), (985, 40) and (981, 55).

B. World models

The world models were trained offline on the same 2,000 episodes per task[66], within a separate study of offline evaluation metrics; this paper uses only their final checkpoints and none of that study's metrics or analyses. The families are a recurrent state-space model[9]; a joint-embedding predictor conditioned on three frame embeddings[16]; TD-MPC2[12], trained from three stacked frames; and MiniWorld-0.5B[18] fine-tuned in Wan2.2 VAE latents[67], sampled block by block with an autoregressive diffusion-forcing sampler[64]. Only final checkpoints are used. Episodes use initial-state seeds 950,000–950,049, disjoint from any earlier evaluation of these checkpoints, and are shared across conditions within a model. A cell enters pooled estimates if its mean paused return is at least 0.15, a rule fixed before the runs; it excludes JEPA walker.

C. Planner

MPPI[6] uses population 512, 6 iterations, temperature 0.5 and horizon $H = 32$ decisions, with a warm start shifted by the decisions executed, and scores candidates by the negative sum of predicted rewards.

D. Asynchronous schedules

Asynchronous schedules start with one plan from the initial observation with the simulator paused. Each subsequent call receives the current latent and the d actions committed for execution during the call, and its output supplies the next d committed actions. In the asynchronous schedules candidates are $d + H$ decisions long, where H is the planning horizon, so $\mathcal{N}_d$, $\mathcal{A}_d$ and $\mathcal{C}_d$ make identical model calls.

E. Residual-calibrated hedging

Algorithm 1 gives one planning call of residual-calibrated hedging in committed-prefix MPPI; residuals are drawn independently per episode and call.

F. Residual bank

For each model and delay, 512 windows of training episodes are filtered with 16 frames of burn-in. The residual is $r = z_{t+d} - \text{rollout}(z_t, a_{t:t+d})$, using the recorded actions. In the closed-loop runs the anchors were scored one after another (batch size 50). For latency measurements they were scored in one batched call; the two agree to 6×10^{-6}.

G. Learned heteroscedastic variant

For each model and delay, 4,096 training windows give $(\hat{z}_{t+d}, z_t, a_{t:t+d})$ and the residual r. A two-layer network (512 SiLU units) predicts the residual's mean and per-dimension log-variance; inputs and targets are standardised. It is trained with a Gaussian negative log-likelihood and AdamW (learning rate 10^{-3}, weight decay 10^{-4}) for 3,000 steps of batch 256, keeping the weights with the best loss on a 10% validation split. On held-out windows, its 95% intervals covered 85–94% of the residuals. In planning, the first anchor is $\hat{z}$ plus the predicted mean, and the others are draws from the predicted Gaussian.

Algorithm 1. Residual-calibrated hedging in committed-prefix MPPI (one call)

Require: latent z_t, committed actions $c \in \mathbb{R}^{d\times m}$, warm start $\mu \in \mathbb{R}^{H\times m}$, residual bank $\mathcal{R}_d$, anchors K, population N, iterations I, temperature λ

$\hat{z} \leftarrow \text{rollout}(z_t, c)$ ▷ once per call
$\hat{z}_1 \leftarrow \hat{z}$; $\hat{z}_k \leftarrow \hat{z} + r_k$, $r_k \sim \mathcal{R}_d$, for $k = 2, \ldots, K$
$\sigma \leftarrow \sigma_0$
for $i = 1, \ldots, I$ **do**
  sample $a^{(n)} \sim \mathcal{N}(\mu, \sigma^2)$, clipped to the action box, for $n = 1, \ldots, N$
  $J_n \leftarrow \frac{1}{K}\sum_k \text{cost}(\hat{z}_k, a^{(n)})$ ▷ all $K \cdot N$ rollouts in one batch
  $w_n \propto \exp(-(J_n - \min_j J_j)/\lambda)$; $\mu \leftarrow \sum_n w_n a^{(n)}$; $\sigma \leftarrow$ weighted standard deviation
end for
return $\mu_{0:d}$ to commit, and μ shifted by d as the next warm start

H. Deployment recipe

1. Measure the latency of the whole control loop, including frame encoding, and set d to its 95th percentile divided by the decision period, rounded up (see 'Fixed-delay evaluation matches real-time control when the delay covers the whole loop' in Results).

2. Record the residual bank for that d from the model's training data. This takes minutes and needs no retraining.

3. Hedge over $K = 4$ anchors at the planner's usual population, scoring them in one batch. Do not reduce the population to make room for anchors (Table II).

4. If several independently trained models are available, an ensemble raises return with or without delay. At longer delays, hedge each member over its own residual bank as well (Table VII).

I. Held-out study and ablations

The mean-correction ablation uses one anchor at the corrected mean. The fixed-spread ablation draws anchors with the global residual standard deviation instead of the predicted one. For the hand-off penalty, a second network of the same form as the learned error model, trained on the same residuals but conditioned only on $(z_t, a_{t:t+d})$, predicts the log-variance of the residual. Each candidate's cost is increased by $\lambda\,\text{sd}(J)\,u$, where u is the within-population z-score of the predicted log-variance of the next call's anchor. The prediction is taken from the corrected anchor and the candidate's first d free actions, which are the actions it would commit. The risk penalty is the anchor-averaged cost plus λ times its standard deviation over anchors. Each λ was chosen from $\{0.25, 0.5, 1.0\}$ by pooled return on the seed-0 models, using 20 episodes from seeds 970,000–970,019, by a rule fixed before the runs.

J. Ensemble and particle baselines

DATS[31] pushes particles from an ensemble of probabilistic dynamics models through the committed actions and averages each candidate's cost over them. Our ensemble is the three independently trained models of each family–task cell, the same checkpoints used individually in every other comparison. Models trained from different seeds have incompatible latent spaces, so a particle cannot move between members from one step to the next, as in the TS-1 propagation used by DATS. Each member instead keeps one particle for the whole rollout, the TS-∞ propagation of PETS[10]. At every decision, each member filters its own latent from the same frames. At each call, each member rolls the committed actions through its own dynamics and scores the shared candidates with its own reward head, and a candidate's cost is the mean over the members. Members are evaluated one after another.

The population-matched ensemble uses population 512 and 6 iterations, the setting of committed-prefix planning and RCH. The latency-matched ensemble uses 2 iterations, so that a call makes the same number of sequential model rollouts as one model at 6 iterations: three members times two iterations. A further exploratory arm combines the two methods: each member hedges over four anchors drawn from its own residual bank.

The particle baseline applies only to the RSSM, the one family with a stochastic transition. Anchor 1 is the prediction, as in RCH. Anchors 2–4 are sampled from the model's own stochastic transition through the committed actions instead of from the residual bank.

All baseline arms use $d \in \{4, 8\}$ and the same initial states and planner noise as the other arms. Two post-hoc checks, each written into the study plan after the primary result and before its runs, complete the comparison. The first runs committed-prefix planning and RCH at 2 iterations on all 24 models, to test whether the smaller number of iterations explains the latency-matched result. The second runs the ensemble with the simulator paused, replanning every 1, 4 and 8 decisions at 6 and 2 iterations. This gives the ensemble's gain without delay and its own mismatch loss $R(\mathcal{P}_d) - R(\mathcal{X}_d)$, measured against the paused ensemble. For a contrast with the ensemble, each cell contributes the paired differences between the

ensemble's return and each single model's return on the same initial state. The bootstrap resamples models and episodes as elsewhere. Each cell has only one ensemble, so the intervals contain no between-ensemble variation.

K. Diagnosis on held-out windows

For each seed-0 model, 64 test windows and 256 calibration windows are drawn from held-out episodes. For each window, plans are made from the true filtered latent z_{t+d}, from the predicted latent $\hat{z}_{t+d}$, and from perturbed anchors, all with the same sampling noise. Regret is computed with the model's cost from z_{t+d}. The candidate corrections are a first-order update along the eight leading principal directions of calibration residuals, using central differences; removal of the mean residual; nearest-anchor selection among 16 anchors in whitened residual coordinates; and anchor-averaged scoring with 4 or 16 anchors.

L. Latency measurement

Latency is the wall-clock time of one call for one environment, from the latent being available to the committed actions being on the host, with CUDA synchronised at both ends. Reference measurements run on a node held exclusively: 20 warm-up and 200 timed calls per configuration for the latent families, and 2 warm-up and 5 timed calls for the diffusion model, from latents encoded from held-out frames. Hedged-call latencies (Table VI) use 10 warm-up and 50 timed calls on a shared node.

M. Real-time check

A simulator process steps one episode on a real-time clock. The planner process reads the frames as they arrive, filters them with the model, and writes plans into a shared action buffer. If no action has been written for a decision, the simulator holds the previous action and records an underrun; no underruns occurred. Before each run, 30 timed calls of the planner after 10 warm-up calls, without frame filtering, give the 95th-percentile latency. The committed prefix length is set to that latency divided by the decision period, rounded up, and the measurement is repeated once with the resulting prefix. Plan actions for decisions that have already executed when a call returns are dropped. Each configuration ran 30 episodes from the first 30 initial states of the fixed-delay runs, on a V100 node held exclusively.

N. Statistics and reproducibility

All contrasts are paired within model and task through shared initial states and planner noise streams. Pooled estimates weight cells equally. Intervals are 95% percentile intervals from a hierarchical bootstrap[68] that resamples models within each cell and episodes within each model (10,000 resamples), or episodes only in the single-model pilot. Multiple contrasts are corrected with Holm's procedure[69]. Confirmatory contrasts are tested one-sided at $\alpha = 0.05$ after Holm correction; each one-sided p value is the share of bootstrap resamples of the pooled contrast at or below zero. The bootstrap makes no normality assumption about episode returns. Numbers after $\pm$ are standard errors of the mean (s.e.m.) over episodes. The pilot decision rule, the replication's confirmatory contrasts, the held-out study's tuning rule and contrasts, and their Holm corrections were written before the respective runs (see 'Pre-specified hypotheses and decision rules'). In the held-out study, contrasts against committed-prefix planning and time-aligned execution were secondary and are reported without correction. The comparison with ensemble hedging claimed no direction, so its contrasts are two-sided, with $p = 2\min(P_{\le 0}, P_{\ge 0})$ over bootstrap resamples. Its secondary contrasts and post-hoc checks are reported without correction.

O. Pre-specified hypotheses and decision rules

See Supplemental Material [70] for supplementary notes on the pre-specified decision rules and compute.

Each rule below was written into the study plan before the runs it concerns. The full study design is `EXPERIMENT_PLAN_SIDE.md` in the code release.

- **Study 1 (4 October 2026).** H1: committed-prefix minus time-aligned return > 0 at each of $d = 2, 4, 8$, pooled over cells above the 0.15 floor. Result: $p = 0.050$, not supported. H2: committed-prefix removes at least half of the time-aligned mismatch loss at $d = 4, 8$. Result: not supported (27–31%).
- **Hedging pilot (5 October 2026, before its runs).** Contrast $R(K = 16) - R(K = 1)$ at $d = 2, 4, 8$ on one model per cell. Rule: proceed if the 95% interval lies above 0 at two or more delays; fall back if it includes 0 at all three. Result: proceed ($d = 8$ and, marginally, $d = 4$).
- **Replication (5 October 2026, before its runs).** Over 24 models, $R(K = 16) - R(K = 1)$ at $d = 4$ and $d = 8$, one-sided, Holm-corrected. Secondary: $d = 2$, and $K = 4$ against $K = 1$. Result: supported at $d = 8$ (Holm $p = 0.0014$), not at $d = 4$ ($p = 0.19$).
- **Equal latency (5 October 2026, before its runs).** $R(K = 4, N = 512) - R(K = 1, N = 512)$

and $R(K = 16, N = 128) - R(K = 1, N = 512)$ at $d = 4, 8$, Holm over the four contrasts. Result: $K = 4$ supported at both delays (Holm $p = 0.035$, 0.0004); $K = 16$ at $N = 128$ worse at both.

- **Held-out method study (6 October 2026, before its runs).** Tuning of the hand-off and risk penalty strengths on the seed-0 models with separate episodes, by a fixed rule (chosen: $\lambda_h = 0.25$, $\lambda_r = 0.5$; the hand-off penalty lowered tuning return at every strength). Primary contrasts on the 16 seed-1/seed-2 models at $d = 4, 8$, Holm over eight:
  - C1: learned error model vs residual bank. Result: not supported (+0.001, +0.011).
  - C2: hand-off penalty vs none. Result: not supported; the penalty lowered return (−0.026, −0.021).
  - C3: hand-off vs risk penalty. Result: not supported (−0.011, +0.006).
  - C4: full variant (learned model with hand-off penalty) vs committed prefix. Result: not supported (−0.001, +0.026).

  Secondary: both hedges beat committed-prefix planning (+0.025/+0.037 for the bank, +0.025/+0.047 for the learned model, at $d = 4/8$).
- **Ensemble and particle baselines (11 October 2026, before their runs).** RCH ($K = 4$) against a three-model ensemble at matched population and at matched latency, $d = 4, 8$, two-sided, Holm over four. Results: RCH below the latency-matched ensemble at $d = 4$ (−0.042, Holm $p = 0.0008$); the other three differences favoured the ensemble with intervals including zero. Secondary: the ensemble against committed-prefix planning, the ensemble with RCH, and particle anchors (Table VII and main text). Two post-hoc checks followed, each written into the plan before its runs: RCH at 2 iterations, and the paused ensemble (Methods, 'Ensemble and particle baselines').

P. Compute

Studies 1–3 and the hedging studies used about 30 GPU-hours (Study 1 and 2 returns), 15 (pilot), 30 (replication), 8 (equal latency), 46 (diffusion) and 13 (ensemble and particle baselines, with their post-hoc checks and timing), plus timing; no training. The ensemble reuses the three models trained for each cell.

Q. Use of an LLM

During preparation of this revision, OpenAI Codex, an LLM-based coding assistant, was used to apply author-directed wording changes to the abstract's latency statement and to adjust LaTeX pagination. For these revision tasks, it was not used to generate or analyse study data, design experiments or interpret results.

DATA AVAILABILITY

The data supporting the findings of this study are available at `https://github.com/rxailab/residual-calibrated-hedging`.

CODE AVAILABILITY

The delay-in-decisions schedules, residual-calibrated hedging planner, delay-error model and analysis scripts are available at `https://github.com/rxailab/residual-calibrated-hedging`.

ACKNOWLEDGMENTS

This work was supported in part by the UK EPSRC under Grant EP/P009727/2, and the Leverhulme Trust under Grant RF-2019-492.

AUTHOR CONTRIBUTIONS

R.X. led the study, including methodology, software, experiments, analysis and manuscript preparation. R.J. supervised the project and contributed to manuscript review and editing. Both authors read and approved the final manuscript.

COMPETING INTERESTS

The authors declare no competing interests.

[1] D. Ha and J. Schmidhuber, Recurrent world models facilitate policy evolution, Advances in Neural Information Processing Systems **31**, 2451 (2018).
[2] C. Finn and S. Levine, Deep visual foresight for planning robot motion, IEEE International Conference on Robotics and Automation (ICRA) , 2786 (2017).
[3] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, Constrained model predictive control: Stability and optimality, Automatica **36**, 789 (2000).

[4] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. (Nob Hill Publishing, 2017).

[5] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, Aggressive driving with model predictive path integral control, IEEE International Conference on Robotics and Automation (ICRA) , 1433 (2016).

[6] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, Information-theoretic model predictive control: Theory and applications to autonomous driving, IEEE Transactions on Robotics **34**, 1603 (2018).

[7] R. Rubinstein, The cross-entropy method for combinatorial and continuous optimization, Methodology and Computing in Applied Probability **1**, 127 (1999).

[8] C. Pinneri, S. Sawant, S. Blaes, J. Achterhold, J. Stueckler, M. Rolinek, *et al.*, Sample-efficient cross-entropy method for real-time planning, Proceedings of the 2020 Conference on Robot Learning, PMLR **155**, 1049 (2021).

[9] D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson, Learning latent dynamics for planning from pixels, Proceedings of the 36th International Conference on Machine Learning, PMLR **97**, 2555 (2019).

[10] K. Chua, R. Calandra, R. McAllister, and S. Levine, Deep reinforcement learning in a handful of trials using probabilistic dynamics models, Advances in Neural Information Processing Systems **31** (2018).

[11] N. Hansen, H. Su, and X. Wang, Temporal difference learning for model predictive control, Proceedings of the 39th International Conference on Machine Learning, PMLR **162**, 8387 (2022).

[12] N. Hansen, H. Su, and X. Wang, TD-MPC2: Scalable, robust world models for continuous control, International Conference on Learning Representations (ICLR) , 47376 (2024).

[13] J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, *et al.*, Mastering Atari, Go, Chess and Shogi by planning with a learned model, Nature **588**, 604 (2020).

[14] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, Dream to control: Learning behaviors by latent imagination, International Conference on Learning Representations (ICLR) (2020).

[15] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, Mastering diverse control tasks through world models, Nature **640**, 647 (2025).

[16] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestriero, LeWorldModel: Stable end-to-end joint-embedding predictive architecture from pixels, arXiv preprint arXiv:2603.19312 (2026).

[17] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, DINO-WM: World models on pre-trained visual features enable zero-shot planning, Proceedings of the 42nd International Conference on Machine Learning, PMLR **267**, 79115 (2025).

[18] Y. Zhao, R. Zheng, H. Guo, Y. Yan, J. Zhang, and J. Chen, MiniWorld: Democratizing the training of video world models from scratch, arXiv preprint arXiv:2608.01127 (2026).

[19] J. Wu, S. Yin, N. Feng, X. He, D. Li, J. Hao, *et al.*, iVideoGPT: Interactive VideoGPTs are scalable world models, Advances in Neural Information Processing Systems **37**, 68082 (2024).

[20] E. Alonso, A. Jelley, V. Micheli, A. Kanervisto, A. Storkey, T. Pearce, *et al.*, Diffusion for world modeling: Visual details matter in Atari, Advances in Neural Information Processing Systems **37**, 58757 (2024).

[21] NVIDIA, Cosmos world foundation model platform for physical AI, arXiv preprint arXiv:2501.03575 (2025).

[22] K. V. Katsikopoulos and S. E. Engelbrecht, Markov decision processes with delays and asynchronous cost collection, IEEE Transactions on Automatic Control **48**, 568 (2003).

[23] S. Ramstedt and C. Pal, Real-time reinforcement learning, Advances in Neural Information Processing Systems **32** (2019).

[24] T. Xiao, E. Jang, D. Kalashnikov, S. Levine, J. Ibarz, K. Hausman, and A. Herzog, Thinking while moving: Deep reinforcement learning with concurrent control, International Conference on Learning Representations (ICLR) (2020).

[25] Y. Bouteiller, S. Ramstedt, G. Beltrame, C. Pal, and J. Binas, Reinforcement learning with random delays, International Conference on Learning Representations (ICLR) (2021).

[26] M. Riemer, G. Subbaraj, G. Berseth, and I. Rish, Enabling realtime reinforcement learning at scale with staggered asynchronous inference, International Conference on Learning Representations (ICLR) , 39073 (2025).

[27] P. Thodoroff, W. Li, and N. D. Lawrence, Benchmarking real-time reinforcement learning, Proceedings of Machine Learning Research **181**, 26 (2022).

[28] O. J. M. Smith, Closer control of loops with dead time, Chemical Engineering Progress **53**, 217 (1957).

[29] R. Findeisen and F. Allgöwer, Computational delay in nonlinear model predictive control, IFAC Proceedings Volumes **37**, 427 (2004).

[30] D. Dirckx, M. Bos, B. Vandewal, L. Vanroye, J. Swevers, and W. Decré, ASAP-MPC: An asynchronous update scheme for online motion planning with nonlinear model predictive control, Autonomous Robots **49**, 8 (2025).

[31] B. Chen, M. Xu, L. Li, and D. Zhao, Delay-aware model-based reinforcement learning for continuous control, Neurocomputing **450**, 119 (2021).

[32] A. Karamzade, K. Kim, J. Lanier, D. Corsi, and R. Fox, Model-based reinforcement learning under random observation delays, Proceedings of the 8th Annual Learning for Dynamics and Control Conference, PMLR **331**, 98 (2026).

[33] Z. El Asri, I. Laiche, C. Rambour, O. Sigaud, and N. Thome, RT-HCP: Dealing with inference delays and sample efficiency to learn directly on robotic platforms, IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) , 597 (2025).

[34] A. Muppidi, F. Darwish, D. Cope, J. F. Henriques, and J. N. Foerster, Finding the time to think: Learning planning budgets in real-time RL, arXiv preprint arXiv:2606.26463 (2026).

[35] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, Learning fine-grained bimanual manipulation with low-cost hardware, Robotics: Science and Systems XIX (2023).

[36] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, Diffusion policy: Visuomotor policy learning via action diffusion, Robotics: Science and Systems XIX (2023).

[37] K. Black, M. Y. Galliker, and S. Levine, Real-time execution of action chunking flow policies, Advances in Neural

Information Processing Systems **38** (2025).
[38] M. Shukor, D. Aubakirova, F. Capuano, P. Kooijmans, S. Palma, A. Zouitine, M. Aractingi, C. Pascal, M. Russi, A. Marafioti, S. Alibert, M. Cord, T. Wolf, and R. Cadene, SmolVLA: A vision-language-action model for affordable and efficient robotics, arXiv preprint arXiv:2506.01844 (2025).
[39] T. Wang, W. Wang, J. Wang, Q. Wang, W. Huang, and B. Xu, RAPAC-DP: Response-aligned pending-action compensation for diffusion policies under delayed execution, arXiv preprint arXiv:2608.15924 (2026).
[40] Motubrain Team, World action models in real time: An empirical study of smooth execution via asynchronous deployment, arXiv preprint arXiv:2608.01880 (2026).
[41] A. Liao, D.-K. Kim, M. O. Smith, A.-a. Aghamohammadi, and S. Omidshafiei, Delay-aware diffusion policy: Bridging the observation-execution gap in dynamic tasks, arXiv preprint arXiv:2512.07697 (2025).
[42] H. Jiang, Y. Zou, B. Liang, B. Liu, F. Meng, and S. Liu, FutureRTC: Real-time robot execution with anticipatory-conditioned action chunking, arXiv preprint arXiv:2607.24008 (2026).
[43] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, *et al.*, World action models are zero-shot policies, arXiv preprint arXiv:2602.15922 (2026).
[44] M. Janner, J. Fu, M. Zhang, and S. Levine, When to trust your model: Model-based policy optimization, Advances in Neural Information Processing Systems **32** (2019).
[45] N. Lambert, B. Amos, O. Yadan, and R. Calandra, Objective mismatch in model-based reinforcement learning, Proceedings of the 2nd Conference on Learning for Dynamics and Control, PMLR **120**, 761 (2020).
[46] S. Beger, Y. Lin, K. Stanojevic, and S. Hirche, Optimal delay compensation in networked predictive control, arXiv preprint arXiv:2512.11492 (2025).
[47] V. M. Zavala and L. T. Biegler, The advanced-step NMPC controller: Optimality, stability and robustness, Automatica **45**, 86 (2009).
[48] T. Belvedere, M. Ziegltrum, G. Turrisi, and V. Modugno, Feedback-MPPI: Fast sampling-based MPC via rollout differentiation – adios low-level controllers, IEEE Robotics and Automation Letters **11**, 1 (2026).
[49] G. C. Calafiore and M. C. Campi, The scenario approach to robust control design, IEEE Transactions on Automatic Control **51**, 742 (2006).
[50] S. Lucia, T. Finkler, and S. Engell, Multi-stage nonlinear model predictive control applied to a semi-batch polymerization reactor under uncertainty, Journal of Process Control **23**, 1306 (2013).
[51] R. Dyro, J. Harrison, A. Sharma, and M. Pavone, Particle MPC for uncertain and learning-based control, IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2021).
[52] M. Gandhi, B. Vlahov, J. Gibson, G. Williams, and E. A. Theodorou, Robust model predictive path integral control: Analysis and performance guarantees, IEEE Robotics and Automation Letters **6**, 1423 (2021).
[53] B. Serfling, K. Doll, and K. Radkhah-Lens, Chance-constrained MPPI under state and dynamic object prediction uncertainty and the evaluation of collision risk calibration, arXiv preprint arXiv:2605.28330 (2026).
[54] Z. Liu and X. Chen, CLIS-MPPI: Closed-loop model predictive path integral control with initial-state sampling under execution delay, IEEE Conference on Decision and Control (CDC), Honolulu, paper WeAT4.5, to appear (2026).
[55] Y. Tassa, Y. Doron, A. Muldal, T. Erez, Y. Li, D. de Las Casas, D. Budden, A. Abdolmaleki, J. Merel, A. Lefrancq, T. Lillicrap, and M. Riedmiller, DeepMind control suite, arXiv preprint arXiv:1801.00690 (2018).
[56] L. Maes, Q. Le Lidec, L. Facury, N. Massaudi, A. Chaurasia, F. Capuano, R. Gao, T. Gillin, D. Haramati, D. Scieur, Y. LeCun, and R. Balestriero, stable-worldmodel: A platform for reproducible world modeling research and evaluation, arXiv preprint arXiv:2605.21800 (2026).
[57] J. Zhang, M. Jiang, N. Dai, T. Lu, A. Uzunoglu, S. Zhang, *et al.*, World-In-World: World models in a closed-loop world, International Conference on Learning Representations (ICLR) , 55660 (2026).
[58] V. J. Reddi, C. Cheng, D. Kanter, P. Mattson, *et al.*, MLPerf inference benchmark, ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA) , 446 (2020).
[59] J. Song, C. Meng, and S. Ermon, Denoising diffusion implicit models, International Conference on Learning Representations (ICLR) (2021).
[60] Z. Lin, Z. Sun, S. Chen, X. Chen, and R. Zhao, Accelerating multi-modal LLM gaming performance via input prediction and mishit correction, arXiv preprint arXiv:2512.17250 (2025).
[61] M. Diehl, H. G. Bock, and J. P. Schlöder, A real-time iteration scheme for nonlinear optimization in optimal feedback control, SIAM Journal on Control and Optimization **43**, 1714 (2005).
[62] O. Melon, R. Orsolino, D. Surovik, M. Geisert, I. Havoutis, and M. Fallon, Receding-horizon perceptive trajectory optimization for dynamic legged locomotion with learned initialization, IEEE International Conference on Robotics and Automation (ICRA) , 9805 (2021).
[63] Y. Cheng, X. Ma, X. Wang, M. Yang, R. Su, H. Liu, X. Chen, S. Li, and Q. Li, AdaReP: Adaptive re-planning under model mismatch for neural world-model predictive control, Artificial Neural Networks and Machine Learning – ICANN 2026, Lecture Notes in Computer Science **17093**, 216 (2026).
[64] B. Chen, D. M. Monso, Y. Du, M. Simchowitz, R. Tedrake, and V. Sitzmann, Diffusion forcing: Next-token prediction meets full-sequence diffusion, Advances in Neural Information Processing Systems **37**, 24081 (2024).
[65] E. Todorov, T. Erez, and Y. Tassa, MuJoCo: A physics engine for model-based control, IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) , 5026 (2012).
[66] R. Xia, MetricTrust: experiment code and diffusion planning pilot records, Repository snapshot ee9cc0d, `https://github.com/rxailab/MetricTrust/tree/ee9cc0d0bd5362e0a0c563a1b63da7c02d86927a` (2026).
[67] Team Wan, A. Wang, B. Ai, B. Wen, *et al.*, Wan: Open and advanced large-scale video generative models, arXiv preprint arXiv:2503.20314 (2025).
[68] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap* (Chapman & Hall, 1993).
[69] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics **6**, 65 (1979).
[70] See Supplemental Material at [URL will be inserted by publisher] for decision rules and compute.